ทำในส่วนของ start() -> stop() ->destroy() ต่อไป โดยหากผู้ใช้ต้องการทำงานที่ต้องการสามารถ Overide method เหล่านี้ได้ กรณีเรียกในลักษณะ Application โปรแกรมจะเรียก main() เป็นอันดับแรกซึ่งจากตัวอย่างใน main มีการสร้าง applet แต่ยังไม่แสดงอะไรที่หน้าจอ หากต้องการให้มีปุ่มที่หน้าจอ หลังจากสร้าง object ของ applet เรียบร้อยให้ทำการเรียก method init() เพื่อทำงานต่อไป โดยใช้คำสั่ง applet.init()

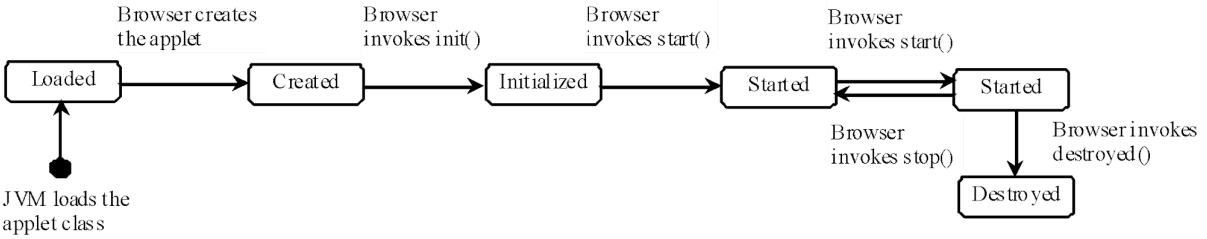
1.3 อธิบายผลลัพธ์ และลำดับขั้นตอนการทำงานของ Applet ต่อไปนี้ method ใดถูกเรียกเป็นอันดับแรก พร้อมทั้งเขียนวงจรการทำงานของ Applet

```
import javax.swing.*;

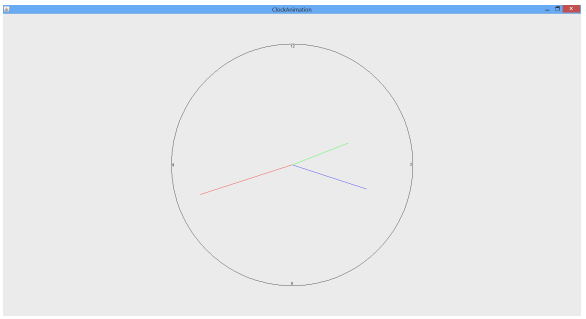
public class Test extends JApplet {
  public Test() {
      System.out.println("Default constructor is invoked");
  }

  public void init() {
      System.out.println("Init method is invoked");
  }
}
```

ผลลัพธ์จากโปรแกรมเมื่อรันผ่านเว็บบราวเซอร์จะแสดงข้อความ Default constructor is invoked โดยลำดับการทำงานของโปรแกรม จะเรียก Constructor แบบไม่มีอาร์กิวเม้นต์ ของคลาสเพื่อทำหน้าที่กำหนดค่าเริ่มต้นเป็นลำดับแรก เมื่อวัตถุถูกสร้างจากคอนสตรั๊กเตอร์เรียบร้อย method init() จะถูกเรียกเป็นลำดับถัดไปดังนั้นผลลัพธ์ที่เกิดขึ้นคือ Init method is invoked หลังจากนั้นจะทำในส่วนของ start() -> stop() ->destroy() ต่อไป โดยหากผู้ใช้ต้องการทำงานที่ต้องการสามารถ overide method เหล่านี้ได้ วงจรการทำงานของ Applet สามารถเขียนได้ดังนี้




2.ให้ศึกษาและทดลองพิมพ์ตัวอย่างการสร้าง Clock Animation จากตัวอย่างต่อไปนี้



```java
import java.awt.event.*;
import javax.swing.*;
import java.awt.*;
import java.util.*;
import javax.swing.Timer;

public class ClockAnimation extends JFrame {
  private StillClock clock = new StillClock();

  public ClockAnimation() {
    add(clock);

    // Create a timer with delay 1000 ms
    Timer timer = new Timer(1000, new TimerListener());
    timer.start();
  }

  private class TimerListener implements ActionListener {
    @Override /** Handle the action event */
    public void actionPerformed(ActionEvent e) {
      // Set new time and repaint the clock to display current time
      clock.setCurrentTime();
      clock.repaint();
    }
  }

  /** Main method */
  public static void main(String[] args) {
    JFrame frame = new ClockAnimation();
    frame.setTitle("ClockAnimation");
    frame.setSize(200, 200);
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
  }
}


 class StillClock extends JPanel {
  private int hour;
  private int minute;
  private int second;

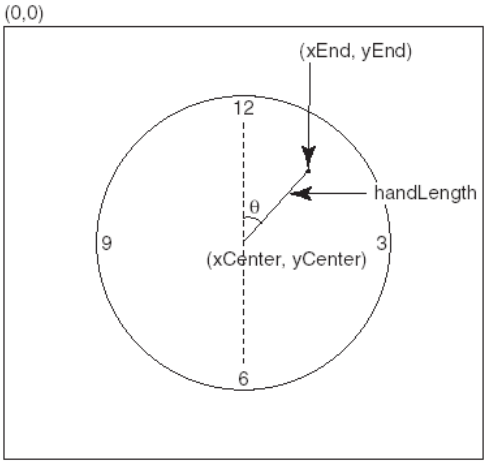
  /** Construct a default clock with the current time*/
  public StillClock() {
    setCurrentTime();
  }

  /** Construct a clock with specified hour, minute, and second */
  public StillClock(int hour, int minute, int second) {
```

```
51      this.hour = hour;
52      this.minute = minute;
53      this.second = second;
54    }
55
56    /** Return hour */
57    public int getHour() {
58      return hour;
59    }
60
61    /** Set a new hour */
62    public void setHour(int hour) {
63      this.hour = hour;
64      repaint();
65    }
66
67    /** Return minute */
68    public int getMinute() {
69      return minute;
70    }
71
72    /** Set a new minute */
73    public void setMinute(int minute) {
74      this.minute = minute;
75      repaint();
76    }
77
78    /** Return second */
79    public int getSecond() {
80      return second;
81    }
82
83    /** Set a new second */
84    public void setSecond(int second) {
85      this.second = second;
86      repaint();
87    }
88
89
90  @Override /** Draw the clock */
91    protected void paintComponent(Graphics g) {
92      super.paintComponent(g);
93
94      // Initialize clock parameters
95      int clockRadius =
96        (int)(Math.min(getWidth(), getHeight()) * 0.8 * 0.5);
97      int xCenter = getWidth() / 2;
98      int yCenter = getHeight() / 2;
99
100     // Draw circle
101     g.setColor(Color.black);
102     g.drawOval(xCenter - clockRadius, yCenter - clockRadius,
103       2 * clockRadius, 2 * clockRadius);
104     g.drawString("12", xCenter - 5, yCenter - clockRadius + 12);
105     g.drawString("9", xCenter - clockRadius + 3, yCenter + 5);
106     g.drawString("3", xCenter + clockRadius - 10, yCenter + 3);
107     g.drawString("6", xCenter - 3, yCenter + clockRadius - 3);
108
109    // Draw second hand
110     int sLength = (int)(clockRadius * 0.8);
111     int xSecond = (int)(xCenter + sLength *
112       Math.sin(second * (2 * Math.PI / 60)));
```

```
113     int ySecond = (int)(yCenter - sLength *
114       Math.cos(second * (2 * Math.PI / 60)));
115     g.setColor(Color.red);
116     g.drawLine(xCenter, yCenter, xSecond, ySecond);
117
118
119 // Draw minute hand
120     int mLength = (int)(clockRadius * 0.65);
121     int xMinute = (int)(xCenter + mLength *
122       Math.sin(minute * (2 * Math.PI / 60)));
123     int yMinute = (int)(yCenter - mLength *
124       Math.cos(minute * (2 * Math.PI / 60)));
125     g.setColor(Color.blue);
126     g.drawLine(xCenter, yCenter, xMinute, yMinute);
127
128     // Draw hour hand
129     int hLength = (int)(clockRadius * 0.5);
130     int xHour = (int)(xCenter + hLength *
131       Math.sin((hour % 12 + minute / 60.0) * (2 * Math.PI / 12)));
132     int yHour = (int)(yCenter - hLength *
133       Math.cos((hour % 12 + minute / 60.0) * (2 * Math.PI / 12)));
134     g.setColor(Color.green);
135     g.drawLine(xCenter, yCenter, xHour, yHour);
136   }
137
138   public void setCurrentTime() {
139     // Construct a calendar for the current date and time
140     Calendar calendar = new GregorianCalendar();
141
142     // Set current hour, minute and second
143     this.hour = calendar.get(Calendar.HOUR_OF_DAY);
144     this.minute = calendar.get(Calendar.MINUTE);
145     this.second = calendar.get(Calendar.SECOND);
146   }
147
148   @Override
149   public Dimension getPreferredSize() {
150     return new Dimension(200, 200);
151   }
152 }
```

## 2.1 อธิบายหลักการของการสร้างเข็มนาฬิกาแต่ละอันมีหลักการสร้างอย่างไร




sin() และ Math.cos() ในการคำนวณจุดปลายเกจุดศูนย์กลาง (xCenter, yCenter) และความ

ารเพิ่มเหตุการณ์ที่ทำให้นาฬิกาเดินได้

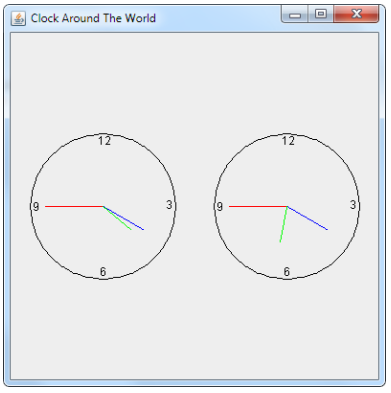
ลง ไว้ใน javax.swing.Timer กำหนด delay = 1000 ms (1 วินาที)

**ลงทะเบียน** `ActionListener` **ให้กับ** `Timer`

**ในเมธอด** `actionPerformed` **เรียก** `setCurrentTime()` **เพื่อดึงเวลาปัจจุบัน แล้วสั่ง** `repaint()`

**สั่ง** `timer.start()` **ให้เวลาเริ่มทำงาน**

2.3 **สร้างนาฬิกาตามเวลาของประเทศไทยและตามเวลาของประเทศญี่ปุ่น ดังรูปต่อไปนี้**




## 3. ศึกษาและทดลองรันโปรแกรมพร้อมคำอธิบายการทำงานของโปรแกรมแต่ละส่วนต่อไปนี้

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

ทำการ Import คลาสและเมธอดต่าง ๆ ที่จำเป็นสำหรับการประมวลผล

```
public class Exercise extends JFrame {
  public Exercise() {
    add(new RaceCar());
  }
```

สร้างคลาส Exercise ที่มีลักษณะเป็นเฟรมโดยการสืบทอดจากคลาส JFrame กำหนดคอนสตรักเตอร์ของคลาส Exercise ให้สร้างวัตถุใหม่ขึ้นมาจากคลาส RaceCar โดยการเรียกใช้คอนสตรักเตอร์ของ Racecar ด้วยคำสั่ง `new RaceCar()`

จากนั้นให้นำวัตถุที่สร้างจากคลาส RaceCar เพิ่มเข้าไปที่คลาส Exercise ที่มีลักษณะเป็น Frame ด้วยคำสั่ง `add(new RaceCar());`

```
  public static void main(String[] args) {
    Exercise frame = new Exercise();
    frame.setTitle("Exercise");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(200, 100);
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setVisible(true);
  }
}
```

`Exercise frame = new Exercise();` ทำการสร้างวัตถุจากคลาส `Exercise`

`frame.setTitle("Exercise");` กำหนดข้อความที่ส่วน Title ของเฟรม เป็น “Excercise”

`frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);` กำหนดให้สามารถปิดโปรแกรมโดยการกดปุ่มเครื่องหมายกากบาทบนเฟรมได้

`frame.setSize(200, 100);` กำหนดขนาดความกว้างของเฟรมให้มีขนาด 200 * 100

`frame.setLocationRelativeTo(null);` กำหนดให้เมื่อประมวลผลชุดคำสั่งให้แสดงเฟรมที่ตำแหน่งกลางหน้าจอ

`frame.setVisible(true);` กำหนดให้เฟรมปรากฏที่หน้าจอ

```
class RaceCar extends JPanel {
  private int xBase = 0;
  private Timer timer = new Timer(10, new Listener());
  public RaceCar() {
    timer.start();
    this.setFocusable(true);
    this.addKeyListener(new KeyAdapter() {
     public void keyPressed(KeyEvent e) {
        if (e.isControlDown() && e.getKeyCode() == 61) {
         if (timer.getDelay() > 5)
          timer.setDelay(timer.getDelay() - 5);
        }
      else if (e.isControlDown() && e.getKeyCode() == 45)
        timer.setDelay(timer.getDelay() + 1);
     }
    });
  }
```

-สร้างคลาส `RaceCar` ที่มีลักษณะเป็น Panel โดยการสืบทอดจากคลาส `JPanel` ซึ่งเป็นบริเวณที่จะใช้วาดรถ

-กำหนดตัวแปร `xBase` มีค่าเป็น 0 ซึ่งเป็นตัวแปรสำหรับเก็บค่าเริ่มต้นของตัวรถ

-สร้างวัตถุจากคลาส Timer และคอยตรวจจับการเกิดเหตุการณ์กับวัตถุ timer โดยการลงทะเบียนการดักจับเหตุการณ์ให้กับ timer โดยใช้คำสั่ง `Timer timer = new Timer(10, new Listener());`

- วัตถุที่จะทำหน้าที่ตรวจจับการเกิดเหตุการณ์และคอยตอบสนองเหตุการณ์ที่เกิดขึ้นคือวัตถุที่สร้างจากคลาสที่ทำการ Implements อินเตอร์เฟส ActionListener โดยเมื่อทำการสร้างคลาสดังกล่าวขึ้นให้ทำการ override เมธอด actionPerformed()

-กำหนดคอนสตรักเตอร์ของคลาส `RaceCar` โดยเมื่อเริ่มสร้างวัตถุจากคลาส `RaceCar` โปรแกรมจะสั่งให้เวลาจากวัตถุ timer เริ่มเดินโดยการใช้ `timer.start();` ซึ่งเมื่อเวลาเริ่มเดินจะทำให้วัตถุที่สร้างจากคลาส `Listener` คอยตอบสนองเหตุการณ์ที่เกิดขึ้นโดยการทำการวาดรถที่ตำแหน่งต่าง ๆ ตามเวลาที่เปลี่ยนไป

-กำหนดให้ Panel สามารถตรวจสอบการกดปุ่มจากคีย์บอร์ดได้โดยการลงทะเบียนการดักจับเหตุการณ์ผ่านคลาสที่มีลักษณะเป็น anonymous class โดยการใช้คำสั่ง `addKeyListener()` โดยวัตถุที่จะทำหน้าที่ตรวจจับการเกิดเหตุการณ์การกดปุ่มและคอยตอบสนองเหตุการณ์ที่เกิดขึ้นคือวัตถุที่สร้างจากคลาส `KeyAdapter` โดยเมื่อทำการสร้างคลาสดังกล่าวขึ้นให้ทำการ override เมธอด `keyPressed ( )` เพื่อตรวจสอบเหตุการณ์ของการกดปุ่ม โดยเหตุการณ์ที่เกิดขึ้นกับปุ่มคือ

1. ปุ่ม Ctrl และ ปุ่ม - ตรวจสอบได้จาก `(e.isControlDown() && e.getKeyCode() == 61)` จะทำให้การแสดงผลของรถช้าลง โดยการใช้คำสั่ง `timer.setDelay(timer.getDelay() - 5);`

2. ปุ่ม Ctrl และ ปุ่ม + ตรวจสอบได้จาก `(e.isControlDown() && e.getKeyCode() == 45)` จะทำให้การแสดงผลของรถเร็วขึ้น โดยการใช้คำสั่ง `timer.setDelay(timer.getDelay() + 1);`

```
class Listener implements ActionListener {
  public void actionPerformed(ActionEvent e) {
   repaint();
  }
}
```

คลาส `Listener` เป็นคลาสที่จะทำหน้าที่ตรวจจับการเกิดเหตุการณ์และคอยตอบสนองเหตุการณ์ที่เกิดขึ้นโดยเป็นคลาสที่ Implements อินเตอร์เฟส ActionListener โดยเมื่อทำการสร้างคลาสดังกล่าวขึ้นให้ทำการ override เมธอด actionPerformed() ภายในเมธอด actionPerformed() เมื่อเกิดการเปลี่ยนของเวลาจะทำการเรียกใช้เมธอด repaint() เพื่อทำการวาดตำแหน่งของรถใหม่ โดยการทำงานที่เมธอด `paintComponent(Graphics g)`

```
  public void paintComponent(Graphics g) {
    super.paintComponent(g);

    int yBase = getHeight();
    if (xBase > getWidth())
      xBase = -20;
    else
      xBase += 1;
```

เมธอด `paintComponent(Graphics g)` เป็นเมธอดที่หน้าที่ในการวาดรูปบน Panel โดยใช้คำสั่ง `super.paintComponent(g);` เป็นคำสั่งสำหรับการเคลียร์หน้าจอเก่า และเตรียมพร้อมสำหรับหน้าจอใหม่ ในชุดคำสั่งแรกจะเป็นการเช็คว่าตำแหน่งเริ่มต้นของตัวรถว่าเกินความกว้างของ Panel หรือไม่หาก ไม่เกินจะทำการเพิ่มตำแหน่งของรถไป 1 ตำแหน่ง ในแกน X

```
    g.setColor(Color.BLACK);
    g.fillOval(xBase + 10, yBase - 10, 10, 10);
    g.fillOval(xBase + 30, yBase - 10, 10, 10);
```

วาดล้อรถที่ 1 ที่ตำแหน่งถัดจากตัวรถในแนวแกน X เป็น `xBase + 10` และถัดจากตัวรถในแนวแกน Y เป็น `yBase - 10` โดยความกว้างและความยาวของวงกลมเป็น 10

วาดล้อรถที่ 2 ที่ตำแหน่งถัดจากตัวรถในแนวแกน X เป็น `xBase + 30` และถัดจากตัวรถในแนวแกน Y เป็น `yBase - 10` โดยความกว้างและความยาวของวงกลมเป็น 10

ล้อทั้งสองจะวาดด้วยสีดำจากการใช้คำสั่ง `g.setColor(Color.BLACK);`

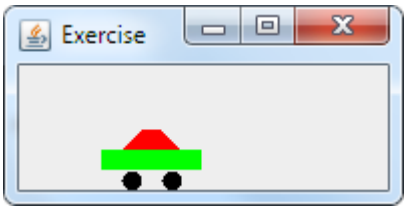
```
    g.setColor(Color.GREEN);
    g.fillRect(xBase, yBase - 20, 50, 10);
```

วาดตัวรถสีเขียวที่มีความกว้างเท่ากับ 50และสูงเท่ากับ 10 ที่ตำแหน่ง `xBase, yBase - 20`

```
    g.setColor(Color.RED);
    Polygon polygon = new Polygon();
    polygon.addPoint(xBase + 10, yBase - 20);
    polygon.addPoint(xBase + 20, yBase - 30);
    polygon.addPoint(xBase + 30, yBase - 30);
    polygon.addPoint(xBase + 40, yBase - 20);
    g.fillPolygon(polygon);
  }
}
```

วาดหลังคารถสีแดงโดยการกำหนดจุดผ่าน Polygon แล้วแสดงผลด้วยการระบายสีบน Polygon ดังกล่าว

## ผลลัพธ์ที่ได้จากการรันโปรแกรมทั้งหมดคือ



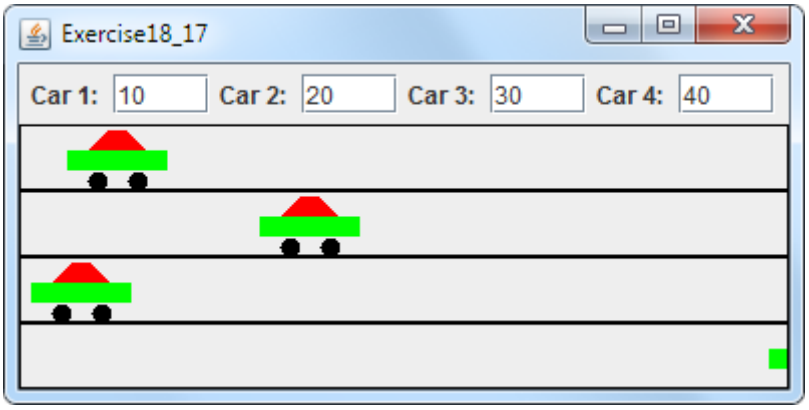

**- ตัวแปร** `xBase`: **เก็บพิกัดแกน X สำหรับวาดตำแหน่งปัจจุบันของตัวรถ**

- `Timer timer = new Timer(10, new Listener());`: **สั่งให้โปรแกรมสร้างลูปเวลาขยับพิกัดรถทุกๆ 10 มิลลิวินาที**

- `addKeyListener(new KeyAdapter())`: **ดักจับคีย์บอร์ด โดยใช้ปุ่ม** `Ctrl +` **เพื่อเพิ่มความเร็ว (**`timer.setDelay(...)`**) และ** `Ctrl -` **เพื่อลดความเร็วรถ**

- `paintComponent(Graphics g)`: **เคลียร์หน้าจอแล้ววาดส่วนประกอบรถใหม่ ได้แก่ ล้อรถสีดำสองวงด้วย** `fillOval`, **ตัวรถสีเขียวด้วย** `fillRect` **และหลังคารถสีแดงด้วย** `fillPolygon`

**4. จากคลาส** Race car **ในปฏิบัติการข้อที่ผ่านมาให้เพิ่มจำนวนรถเป็น 4 คัน จำลองให้เป็นสนามแข่งรถ และสามารถกำหนดความเร็วให้รถแต่ละคันได้**




**5.เขียนโปรแกรมที่แสดงรูปบอลลอนในตำแหน่ง** random **ใน** Panel **ให้ใช้ปุ่มลูกศรซ้ายและลูกศรขวาในการเล็งตำแหน่งของปืนให้ตรงกับบอลลูน ใช้ปุ่มลูกศร** up-arrow **สำหรับยิงลูกกระสุน เมื่อลูกกระสุนถูกบอลลูนให้บอลลูนที่โดนกระสุนหายไป และ** random **บอลลูนที่ตำแหน่งใหม่ขึ้นมา**